



المحاضرة الثالثة:

التعابير والتفاعلية

Expressions and Interactivity

Topics for Lecture 3

3.1 The cin Object

3.2 Mathematical Expressions

3.3 When You Mix Apples and Oranges: Type Conversion

3.4 Overflow and Underflow

3.5 Type Casting

3.6 Multiple Assignment and Combined Assignment

3.7 Formatting Output

3.8 Working with Characters and string Objects

3.9 More Mathematical Library Functions

3.10 Focus on Debugging: Hand Tracing a Program

3.1 The cin Object

المفهوم: يمكن استخدام الكائن cin لقراءة البيانات المكتوبة من لوحة المفاتيح.

تمامًا كما يُعتبر cout كائن الإخراج القياسي في لغة C++، فإن cin هو كائن الإدخال القياسي. يقوم بقراءة المدخلات من الوحدة الطرفية (أو لوحة المفاتيح) كما هو موضح في البرنامج 3-1.

Program 3-1

```
1 // This program asks the user to enter the length and width of
2 // a rectangle. It calculates the rectangle's area and displays
3 // the value on the screen.
4 #include <iostream>
5 using namespace std;
6
7 int main()
8 {
9     int length, width, area;
10
11     cout << "This program calculates the area of a ";
12     cout << "rectangle.\n";
13     cout << "What is the length of the rectangle? ";
14     cin >> length;
15     cout << "What is the width of the rectangle? ";
16     cin >> width;
17     area = length * width;
18     cout << "The area of the rectangle is " << area << ".\n";
19     return 0;
20 }
```

Program Output with Example Input Shown in Bold

This program calculates the area of a rectangle.
What is the length of the rectangle? **10**
What is the width of the rectangle? **20**
The area of the rectangle is 200.

جمع المدخلات من المستخدم عادةً ما يكون عملية من خطوتين:

1. استخدام كائن cout لعرض رسالة توجيهية (Prompt) على الشاشة.
2. استخدام كائن cin لقراءة القيمة من لوحة المفاتيح.

Figure 3-1 The << and >> operators

```
cout << "What is the length of the rectangle? ";
cin >> length;
```

Think of the << and >> operators as arrows that point in the direction that data is flowing.

```
cout ← "What is the length of the rectangle? ";
cin → length;
```


يؤدي استخدام كائن cin إلى جعل البرنامج ينتظر حتى يتم إدخال البيانات من لوحة المفاتيح والضغط على مفتاح **Enter**. لن يتم تنفيذ أي أسطر أخرى في البرنامج حتى يحصل cin على مدخلاته.

يقوم cin تلقائيًا بتحويل البيانات المقروءة من لوحة المفاتيح إلى نوع البيانات الخاص بالمتغير المستخدم لتخزينها. على سبيل المثال، إذا كتب المستخدم الرقم 10، فسيتم قراءته كحرفين هما '1' و '0'. ومع ذلك، فإن cin ذكي بما يكفي ليعرف أنه يجب تحويل هذه القيمة إلى قيمة عدد صحيح (int) قبل تخزينها في المتغير length.

كما أن cin ذكي بما يكفي ليعرف أن قيمة مثل 10.7 لا يمكن تخزينها في متغير من نوع عدد صحيح (int). إذا أدخل المستخدم قيمة عشرية (فاصلة عائمة) لمتغير عدد صحيح، فإن cin لن يقرأ الجزء الذي يلي الفاصلة العشرية.



NOTE: You must include the <iostream> header file in any program that uses cin.

• Entering Multiple Values

يمكن استخدام كائن cin لجمع قيم متعددة في وقت واحد. انظر إلى البرنامج 2-3، وهو نسخة معدلة من البرنامج 1-3.

Program 3-2

```
1 // This program asks the user to enter the length and width of
2 // a rectangle. It calculates the rectangle's area and displays
3 // the value on the screen.
4 #include <iostream>
5 using namespace std;
6
7 int main()
8 {
9     int length, width, area;
10
11     cout << "This program calculates the area of a ";
12     cout << "rectangle.\n";
13     cout << "Enter the length and width of the rectangle ";
14     cout << "separated by a space.\n";
15     cin >> length >> width;
16     area = length * width;
17     cout << "The area of the rectangle is " << area << endl;
18     return 0;
19 }
```

Program Output with Example Input Shown in Bold

This program calculates the area of a rectangle.
Enter the length and width of the rectangle separated by a space.
10 20
The area of the rectangle is 200

لاحظ أن المستخدم يفصل بين الأرقام بمسافات عند إدخالها. هذه هي الطريقة التي يعرف بها cin مكان بداية ونهاية كل رقم. ولا يهم عدد المسافات المدخلة بين الأرقام الفردية. على سبيل المثال، كان يمكن للمستخدم إدخال:

10 20



NOTE: The **Enter** key is pressed after the last number is entered.

يمكن لـ cin أيضاً قراءة قيم متعددة من أنواع بيانات مختلفة. يوضح هذا البرنامج 3-3.

Program 3-3

```

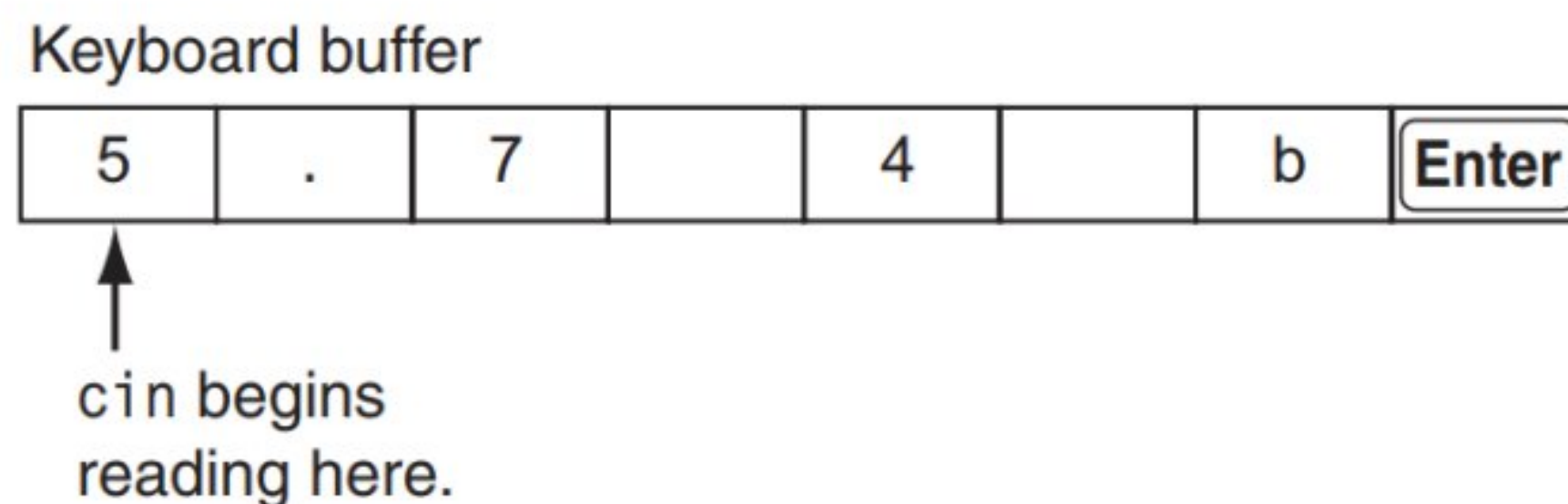
1 // This program demonstrates how cin can read multiple values
2 // of different data types.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     int whole;
9     double fractional;
10    char letter;
11
12    cout << "Enter an integer, a double, and a character: ";
13    cin >> whole >> fractional >> letter;
14    cout << "Whole: " << whole << endl;
15    cout << "Fractional: " << fractional << endl;
16    cout << "Letter: " << letter << endl;
17    return 0;
18 }
```

Program Output with Example Input Shown in Bold

Enter an integer, a double, and a character: **4 5.7 b** **Enter**
 Whole: 4
 Fractional: 5.7
 Letter: b

عندما يقوم المستخدم بكتابة القيم من لوحة المفاتيح، يتم تخزين تلك القيم أولاً في منطقة من الذاكرة تُعرف باسم **مخزن مؤقت لوحة المفاتيح (Keyboard Buffer)** لذلك، عندما يقوم المستخدم بإدخال القيم 5.7 و 4 و b، يتم تخزينها في المخزن المؤقت كما هو موضح في الشكل 3-2.

Figure 3-2 The keyboard buffer



عندما يضغط المستخدم على مفتاح **Enter**، يقوم cin بقراءة القيمة 5 وتخزينها في المتغير whole ولا يقوم بقراءة العلامة العشرية (النقطة) لأن whole هو متغير من نوع عدد صحيح (int). بعد ذلك، يقرأ القيمة 7. ويخزنها في المتغير fractional من نوع double. يتم تخطي المسافة، ثم يتم قراءة القيمة 4 وتخزينها كحرف في المتغير letter. نظرًا لأن عبارة cin هذه تقرأ ثلاثة قيم فقط، يتم ترك الحرف b في المخزن المؤقت لوحة المفاتيح. لذلك، في هذه الحالة، سيخزن البرنامج القيمة 5 في whole، والقيمة 0.7 في fractional، والحرف '4' في letter.

من المهم أن يدخل المستخدم القيم بالترتيب الصحيح.

3.2 Mathematical Expressions

المفهوم: تسمح لغة C++ ببناء تعبيرات رياضية معقدة باستخدام عدة عوامل تشغيل ورموز تجميع. التعبير هو عبارة برمجية لها قيمة. عادةً ما يتكون التعبير من عامل تشغيل (operator) ومعاملاته (operands). انظر إلى العبارة التالية: $sum = 21 + 3$ ؛ نظرًا لأن $21 + 3$ له قيمة، فهو تعبير. يتم تخزين قيمته، وهي 24، في المتغير sum.

فيما يلي بعض العبارات البرمجية حيث يتم إسناد المتغير result بقيمة تعبير:

```
result = x;
result = 4;
result = 15 / 3;
result = 22 * number;
result = sizeof(int);
result = a + b + c;
```

في كل من هذه العبارات، يظهر رقم أو اسم متغير أو تعبير رياضي على الجانب الأيمن من رمز =. يتم الحصول على قيمة من كل من هذه العناصر وتخزينها في المتغير result. هذه كلها أمثلة على تعيين متغير بقيمة تعبير.

يوضح البرنامج 3-4 كيفية استخدام التعبيرات الرياضية مع كائن cout.

Program 3-4

```
1 // This program asks the user to enter the numerator
2 // and denominator of a fraction and it displays the
3 // decimal value.
4
5 #include <iostream>
6 using namespace std;
7
8 int main()
9 {
10     double numerator, denominator;
11
12     cout << "This program shows the decimal value of ";
13     cout << "a fraction.\n";
```


Program 3-4

(continued)

```
14      cout << "Enter the numerator: ";
15      cin >> numerator;
16      cout << "Enter the denominator: ";
17      cin >> denominator;
18      cout << "The decimal value is ";
19      cout << (numerator / denominator) << endl;
20      return 0;
21  }
```

Program Output with Example Input Shown in Bold

This program shows the decimal value of a fraction.

Enter the numerator: **3**

Enter the denominator: **16**

The decimal value is 0.1875



NOTE: The example input for Program 3-4 shows the user entering 3 and 16. Since these values are assigned to double variables, they are stored as the double values 3.0 and 16.0.



NOTE: When sending an expression that consists of an operator to cout, it is always a good idea to put parentheses around the expression. Some advanced operators will yield unexpected results otherwise.

• Operator Precedence

يوضح الجدول 1-3 أولوية عوامل التشغيل الحسابية. العوامل الموجودة في أعلى الجدول لها أولوية أعلى من تلك الموجودة أسفلها.

Table 3-1 Precedence of Arithmetic Operators (Highest to Lowest)

(unary negation) -

* / %

+ -

Table 3-2 Some Simple Expressions and Their Values

Expression	Value
$5 + 2 * 4$	13
$10 / 2 - 3$	2
$8 + 12 * 2 - 4$	28
$4 + 17 \% 2 - 1$	4
$6 - 3 * 2 + 7 - 1$	6

- Associativity**

اتجاهية العامل (Associativity) إما تكون من اليسار إلى اليمين (Left to Right) أو من اليمين إلى اليسار (Right to Left). إذا كان هناك عاملان يشتركان في نفس المعامل ولديهما نفس الأولوية، فإنهما يعملان وفقًا لاتجاهيتهما.

Table 3-3 Associativity of Arithmetic Operators

Operator	Associativity
(unary negation) -	Right to left
* / %	Left to right
+ -	Left to right

- Grouping with Parentheses**

يمكن تجميع أجزاء من التعبير الرياضي باستخدام الأقواس لإجبار بعض العمليات على التنفيذ قبل غيرها.

Table 3-4 More Simple Expressions and Their Values

Expression	Value
$(5 + 2) * 4$	28
$10 / (5 - 3)$	5
$8 + 12 * (6 - 2)$	56
$(4 + 17) \% 2 - 1$	0
$(6 - 3) * (2 + 7) / 3$	9

- **Converting Algebraic Expressions to Programming Statements**

Table 3-5 Algebraic and C++ Multiplication Expressions

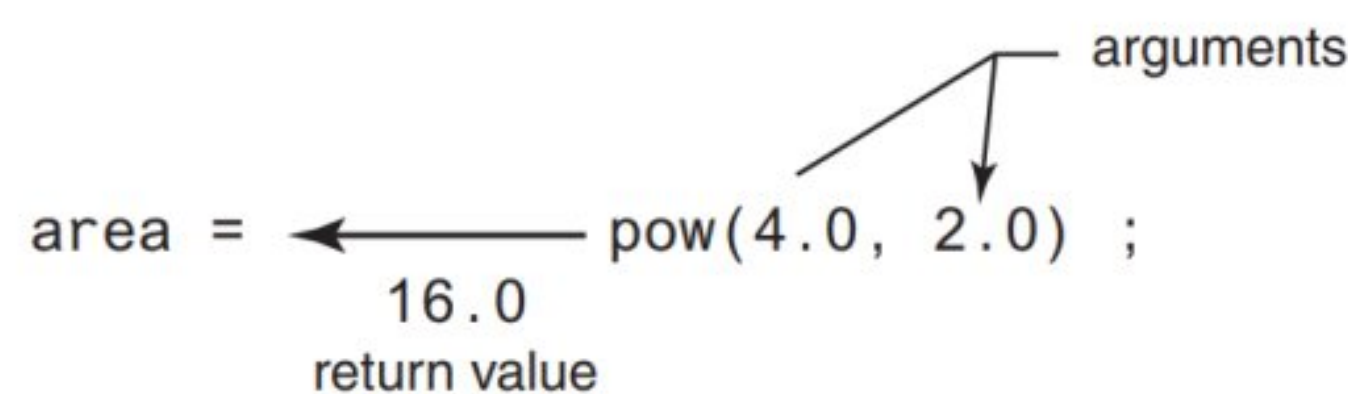
Algebraic Expression	Operation	C++ Equivalent
$6B$	6 times B	$6 * B$
$(3)(12)$	3 times 12	$3 * 12$
$4xy$	4 times x times y	$4 * x * y$

Table 3-6 Algebraic and C++ Expressions

Algebraic Expression	C++ Expression
$y = 3\frac{x}{2}$	$y = x / 2 * 3;$
$z = 3bc + 4$	$z = 3 * b * c + 4;$
$a = \frac{3x + 2}{4a - 1}$	$a = (3 * x + 2) / (4 * a - 1)$

- **No Exponents Please!**

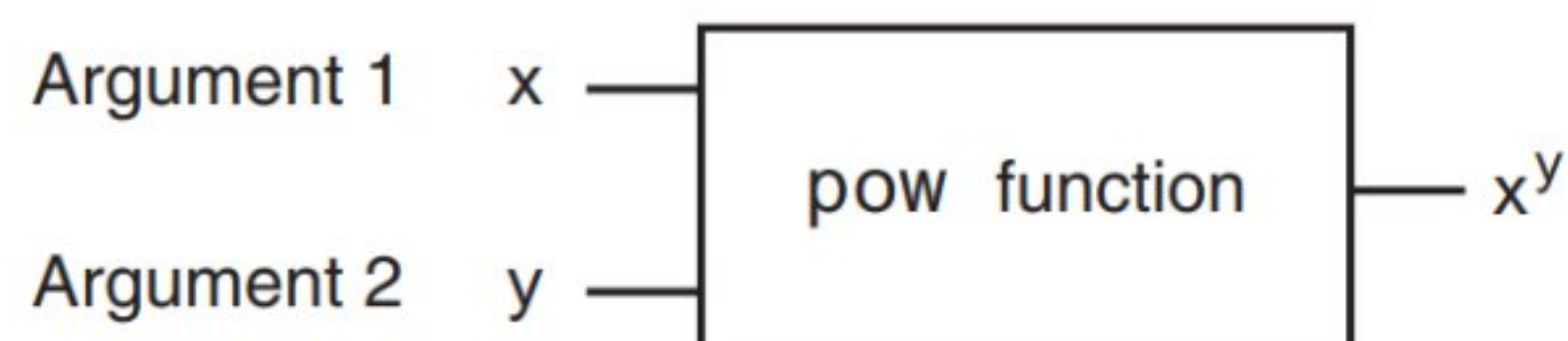
Figure 3-3 The pow function



The statement $\text{area} = \text{pow}(4.0, 2.0)$ is equivalent to the following algebraic statement:

$$\text{area} = 4^2$$

Figure 3-4 The pow function as a “black box”



يقوم البرنامج 3-5 بحل مسألة جبرية بسيطة. يطلب من المستخدم إدخال نصف قطر الدائرة ثم يحسب مساحة الدائرة. والتي يتم التعبير عنها في البرنامج كالتالي:

$\text{area} = \text{PI} * \text{pow}(\text{radius}, 2.0);$

Program 3-5

```
1 // This program calculates the area of a circle.
2 // The formula for the area of a circle is Pi times
3 // the radius squared. Pi is 3.14159.
4 #include <iostream>
5 #include <cmath> // needed for pow function
6 using namespace std;
7
8 int main()
9 {
10     const double PI = 3.14159;
11     double area, radius;
12
13     cout << "This program calculates the area of a circle.\n";
14     cout << "What is the radius of the circle? ";
15     cin >> radius;
16     area = PI * pow(radius, 2.0);
17     cout << "The area is " << area << endl;
18     return 0;
19 }
```

Program Output with Example Input Shown in Bold

```
This program calculates the area of a circle.
What is the radius of the circle? 10 
The area is 314.159
```

3.3 When You Mix Apples and Oranges: Type Conversion

المفهوم: عندما تكون معاملات العامل (operands) من أنواع بيانات مختلفة، تقوم لغة C++ تلقائيًا بتحويلها إلى نفس نوع البيانات. يمكن أن يؤثر ذلك على نتائج التعبيرات الرياضية.

إذا تم ضرب int في float، فما نوع البيانات الذي ستكون عليه النتيجة؟ وماذا لو تم قسمة double على unsigned int؟ هل هناك أي طريقة للتنبؤ بما سيحدث في هذه الحالات؟ الإجابة هي نعم. تتبع لغة C++ مجموعة من القواعد عند إجراء العمليات الرياضية على متغيرات من أنواع بيانات مختلفة. من المهم فهم هذه القواعد لمنع حدوث أخطاء دقيقة في برامجك.

Table 3-7 Data Type Ranking

long double	تمامًا مثل الرتب العسكرية، يتم ترتيب أنواع البيانات. نوع بيانات واحد يتفوق
double	على آخر إذا كان بإمكانه تخزين عدد أكبر. على سبيل المثال، float يتفوق
float	على int. يوضح الجدول 3-7 أنواع البيانات مرتبة حسب رتبته، من الأعلى
unsigned long long int	إلى الأدنى.
long long int	
unsigned long int	
long int	
unsigned int	
int	

استثناء واحد من الترتيب في الجدول 3-7 هو عندما يكون `int` و `long` بنفس الحجم. في هذه الحالة، يتفوق `unsigned int` على `long` لأنه يمكنه تخزين قيمة أعلى.

عندما تعمل لغة `C++` مع عامل تشغيل (`operator`)، تسعى إلى تحويل المعاملات (`operands`) إلى نفس النوع. يُعرف هذا التحويل التلقائي باسم الإكراه النوعي (`Type Coercion`). عندما يتم تحويل قيمة إلى نوع بيانات أعلى، يُقال إنها تمت ترقيتها (`Promoted`). أما تخفيض قيمة (`Demote`) فيعني تحويلها إلى نوع بيانات أدنى. دعونا نلقي نظرة على القواعد المحددة التي تحكم تقييم التعبيرات الرياضية.

القاعدة 1: يتم ترقيّة `char` و `short` و `unsigned short` تلقائيًا إلى `int`.

ستلاحظ أن `char` و `short` و `unsigned short` غير موجودة في الجدول 3-7. وذلك لأنه في أي وقت يتم استخدامها في تعبير رياضي، يتم ترقيتها تلقائيًا إلى `int`. الاستثناء الوحيد من هذه القاعدة هو عندما يحمل `unsigned short` قيمة أكبر مما يمكن أن يحمله `int`. يمكن أن يحدث هذا في الأنظمة التي يكون فيها `short` بنفس حجم `int`. في هذه الحالة، يتم ترقيّة `unsigned short` إلى `unsigned int`.

القاعدة 2: عندما يعمل عامل تشغيل مع قيمتين من نوعي بيانات مختلفين، يتم ترقيّة القيمة ذات الرتبة الأدنى إلى نوع البيانات ذات الرتبة الأعلى. في التعبير التالي، لنفترض أن `years` من نوع `int` وأن `interestRate` من نوع `float`:

$$\text{years} * \text{interestRate}$$

قبل إجراء عملية الضرب، سيتم ترقيّة `years` إلى `float`.

القاعدة 3: عند إسناد القيمة النهائية لتعبير إلى متغير، سيتم تحويلها إلى نوع بيانات ذلك المتغير. في العبارة التالية، لنفترض أن `area` من نوع `long int`، بينما `length` و `width` كلاهما من نوع `int`:

```
area = length * width;
```

نظرًا لأن `length` و `width` كلاهما من نوع `int`، لن يتم تحويلهما إلى أي نوع بيانات آخر. ومع ذلك، سيتم تحويل نتيجة الضرب إلى `long` حتى يمكن تخزينها في `area`.

احذر من المواقف التي ينتج عنها تعيين قيمة عشرية إلى متغير عدد صحيح. إليك مثالًا:

```
int x, y = 4;
float z = 2.7;
x = y * z;
```

في التعبير `y * z`، سيتم ترقيّة `y` إلى `float` وستكون نتيجة الضرب 10.8. ومع ذلك، نظرًا لأن `x` من نوع `int`، سيتم اقتطاع 10.8 وسيتم تخزين 10 في `x`.

• Integer Division

عندما تقوم بقسمة عدد صحيح على عدد صحيح آخر في لغة `C++`، تكون النتيجة دائمًا عددًا صحيحًا. إذا كان هناك باقي، فسيتم تجاهله. على سبيل المثال، في الكود التالي، يتم تعيين القيمة 2.0 للمتغير `parts`:

```
double parts;
parts = 15 / 6;
```

على الرغم من أن 15 مقسومًا على 6 هو في الواقع 2.5، إلا أن الجزء العشري 0.5 من النتيجة يتم تجاهله لأننا نقوم بقسمة عدد صحيح على عدد صحيح. لا يهم أن `parts` تم تعريفه كـ `double` لأن الجزء العشري من النتيجة يتم تجاهله قبل حدوث عملية التعيين.

لكي تُرجع عملية القسمة قيمة عشرية (فاصلة عائمة)، يجب أن يكون أحد المعاملات على الأقل من نوع بيانات عشري. على سبيل المثال، يمكن كتابة الكود السابق كالتالي:


```
double parts;  
parts = 15.0 / 6;
```

في هذا الكود، يتم تفسير القيمة 15.0 كرقم عشري، لذا سترجع عملية القسمة قيمة عشرية. سيتم تعيين القيمة 2.5 إلى parts.

3.4 Overflow and Underflow

المفهوم: عندما يتم تعيين قيمة كبيرة جدًا أو صغيرة جدًا لنطاق نوع بيانات المتغير، يحدث فيضان (Overflow) أو نقص (Underflow) للمتغير.

يمكن أن تحدث مشكلة عندما يتم تعيين قيمة كبيرة جدًا لنوع المتغير. إليك عبارة حيث تكون a و b و c جميعها من نوع short int:

```
a = b * c;
```

إذا كانت قيم b و c كبيرتين بما يكفي، فإن عملية الضرب ستنتج رقمًا أكبر من أن يتم تخزينه في a. للاستعداد لهذا الموقف، كان يجب تعريف a ك int أو long int.

عندما يتم تعيين رقم كبير جدًا لنوع بيانات المتغير، يحدث فيضان (Overflow). وبالمثل، يؤدي تعيين قيمة صغيرة جدًا إلى حدوث نقص (Underflow). يوضح البرنامج 3-7 ما يحدث عندما يفيض أو ينقص عدد صحيح. (النتيجة المعروضة هي من نظام يستخدم أعدادًا صحيحة قصيرة بحجم 2 بايت).

Program 3-7

```
1 // This program demonstrates integer overflow and underflow.  
2 #include <iostream>  
3 using namespace std;  
4  
5 int main()  
6 {  
7     // testVar is initialized with the maximum value for a short.  
8     short testVar = 32767;  
9  
10    // Display testVar.  
11    cout << testVar << endl;  
12  
13    // Add 1 to testVar to make it overflow.  
14    testVar = testVar + 1;  
15    cout << testVar << endl;  
16  
17    // Subtract 1 from testVar to make it underflow.  
18    testVar = testVar - 1;  
19    cout << testVar << endl;  
20    return 0;  
21 }
```

Program Output

```
32767  
-32768  
32767
```

عادةً، عندما يفيض (Overflow) عدد صحيح، تتحول قيمته إلى أقل قيمة ممكنة لنوع البيانات هذا. في البرنامج 3-7، لفت testVar من 32,767 إلى -32,768. عندما تمت إضافة 1 إليها. وعندما تم طرح 1 من testVar، حدث نقص (underflow)، مما تسبب في أن قيمته مرة أخرى تحولت إلى 32,767. لا يتم إعطاء أي تحذير أو رسالة خطأ، لذا كن حذرًا عند العمل بأرقام قريبة من الحد الأقصى أو الأدنى لنطاق الأعداد الصحيحة. إذا حدث فيضان أو نقص، سيستخدم البرنامج الرقم غير الصحيح، وبالتالي سيُنتج نتائج غير صحيحة.

- عندما تفيض أو تنقص المتغيرات ذات الفاصلة العائمة (floating-point)، تعتمد النتائج على كيفية تكوين المترجم (compiler). قد ينتج نظامك برامج تقوم بأي مما يلي:
- إنتاج نتيجة غير صحيحة والاستمرار في التشغيل.
 - طباعة رسالة خطأ والتوقف فوراً عند حدوث فيضان أو نقص في الفاصلة العائمة.
 - طباعة رسالة خطأ والتوقف فوراً عند حدوث فيضان في الفاصلة العائمة، ولكن تخزين 0 في المتغير عند حدوث نقص.
 - منحك خياراً للسلوكيات عند حدوث فيضان أو نقص.
- يمكنك معرفة كيفية تفاعل نظامك من خلال ترجمة وتشغيل البرنامج 3-8.

Program 3-8

```

1 // This program can be used to see how your system handles
2 // floating-point overflow and underflow.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     float test;
9
10    test = 2.0e38 * 1000;    // Should overflow test.
11    cout << test << endl;
12    test = 2.0e-38 / 2.0e38; // Should underflow test.
13    cout << test << endl;
14    return 0;
15 }
```

3.5 Type Casting

المفهوم: التحويل النوعي (قسر النمط) يتيح لك إجراء تحويل يدوي لنوع البيانات.

تعبير التحويل النوعي يتيح لك ترقية أو تخفيض قيمة يدوياً. الصيغة العامة لتعبير التحويل النوعي هي:

`static_cast<DataType>(Value)`

حيث:

- **القيمة (Value):** هي متغير أو قيمة ثابتة ترغب في تحويلها.
 - **نوع البيانات (DataType):** هو نوع البيانات الذي ترغب في تحويل القيمة إليه.
- إليك مثال على كود يستخدم تعبير التحويل النوعي:

```
double number = 3.7;
```

```
int val;
```

```
val = static_cast<int>(number);
```

في هذا المثال:

- يتم تحويل القيمة number من نوع double إلى نوع int باستخدام static_cast.
- النتيجة المخزنة في val ستكون 3 حيث يتم تجاهل الجزء العشري عند التحويل إلى عدد صحيح.

يوضح البرنامج 9-3 مثالاً يتم فيه استخدام تعبير التحويل النوعي لمنع حدوث عملية القسمة الصحيحة (integer division).

Program 3-9

```
1 // This program uses a type cast to avoid integer division.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int books;           // Number of books to read
8     int months;          // Number of months spent reading
9     double perMonth;     // Average number of books per month
10
11     cout << "How many books do you plan to read? ";
12     cin >> books;
13     cout << "How many months will it take you to read them? ";
14     cin >> months;
15     perMonth = static_cast<double>(books) / months;
16     cout << "That is " << perMonth << " books per month.\n";
17     return 0;
18 }
```

Program Output with Example Input Shown in Bold

How many books do you plan to read? **30**
How many months will it take you to read them? **7**
That is 4.28571 books per month.



WARNING! In Program 3-9, the following statement would still have resulted in integer division:

```
perMonth = static_cast<double>(books / months);
```

The result of the expression `books / months` is 4. When 4 is converted to a `double`, it is 4.0. To prevent the integer division from taking place, one of the operands should be converted to a `double` prior to the division operation. This forces C++ to automatically convert the value of the other operand to a `double`.

Program 3-10 further demonstrates the type cast expression.

Program 3-10

```
1 // This program uses a type cast expression to print a character
2 // from a number.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     int number = 65;
9
10    // Display the value of the number variable.
11    cout << number << endl;
12
13    // Display the value of number converted to
14    // the char data type.
15    cout << static_cast<char>(number) << endl;
16    return 0;
17 }
```

Program Output

65
A



NOTE: C++ provides several different type cast expressions. `static_cast` is the most commonly used type cast expression, so we will primarily use it in this book.

3.6 Multiple Assignment and Combined Assignment

المفهوم: التعيين المتعدد (الإسناد المتعدد) يعني تعيين (إسناد) نفس القيمة لعدة متغيرات باستخدام عبارة واحدة.
تسمح لغة C++ بتعيين قيمة واحدة لعدة متغيرات في نفس الوقت. إذا كان البرنامج يحتوي على عدة متغيرات، مثل `a`، `b`، `c`، و `d`، ويحتاج كل متغير إلى تعيين قيمة معينة، مثل 12، يمكن إنشاء العبارة التالية:

`a = b = c = d = 12;`

سيتم تعيين القيمة 12 لكل متغير مذكور في العبارة.
يعمل عامل التعيين (عامل الإسناد) من اليمين إلى اليسار. يتم أولاً تعيين القيمة 12 إلى المتغير `d`، ثم إلى `c`، ثم إلى `b`، وأخيراً إلى `a`.

• Combined Assignment Operators

Table 3-8 (Assume `x = 6`)

Statement	What It Does	Value of <code>x</code> After the Statement
<code>x = x + 4;</code>	Adds 4 to <code>x</code>	10
<code>x = x - 3;</code>	Subtracts 3 from <code>x</code>	3
<code>x = x * 10;</code>	Multiplies <code>x</code> by 10	60
<code>x = x / 2;</code>	Divides <code>x</code> by 2	3
<code>x = x % 4</code>	Makes <code>x</code> the remainder of <code>x / 4</code>	2

يعرض الجدول 9-3 عوامل التعيين المركبة (Combined Assignment Operators)، والمعروفة أيضًا باسم العوامل المركبة (Compound Operators) أو عوامل التعيين الحسابية (Arithmetic Assignment Operators). هذه العوامل تسمح بإجراء عملية حسابية وتعيين النتيجة لمتغير في خطوة واحدة، مما يجعل الكود أكثر إيجازًا وسهولة في القراءة.

Table 3-9 Combined Assignment Operators

Operator	Example Usage	Equivalent to
+=	x += 5;	x = x + 5;
-=	y -= 2;	y = y - 2;
*=	z *= 10;	z = z * 10;
/=	a /= b;	a = a / b;
%=	c %= 3;	c = c % 3;

البرنامج 11-3 يستخدم عوامل التعيين المركبة (Combined Assignment Operators) لتبسيط العمليات الحسابية وتعيين القيم للمتغيرات في خطوة واحدة. هذه العوامل تجعل الكود أكثر إيجازًا وسهولة في القراءة.

Program 3-11

```
1 // This program tracks the inventory of three widget stores
2 // that opened at the same time. Each store started with the
3 // same number of widgets in inventory. By subtracting the
```

Program 3-11 (continued)

```
4 // number of widgets each store has sold from its inventory,
5 // the current inventory can be calculated.
6 #include <iostream>
7 using namespace std;
8
9 int main()
10 {
11     int begInv,    // Beginning inventory for all stores
12         sold,      // Number of widgets sold
13         store1,    // Store 1's inventory
14         store2,    // Store 2's inventory
15         store3;    // Store 3's inventory
16
17     // Get the beginning inventory for all the stores.
18     cout << "One week ago, 3 new widget stores opened\n";
19     cout << "at the same time with the same beginning\n";
20     cout << "inventory. What was the beginning inventory? ";
21     cin >> begInv;
22
23     // Set each store's inventory.
24     store1 = store2 = store3 = begInv;
25
26     // Get the number of widgets sold at store 1.
27     cout << "How many widgets has store 1 sold? ";
28     cin >> sold;
29     store1 -= sold; // Adjust store 1's inventory.
30
31     // Get the number of widgets sold at store 2.
32     cout << "How many widgets has store 2 sold? ";
33     cin >> sold;
34     store2 -= sold; // Adjust store 2's inventory.
35
36     // Get the number of widgets sold at store 3.
37     cout << "How many widgets has store 3 sold? ";
38     cin >> sold;
39     store3 -= sold; // Adjust store 3's inventory.
40
41     // Display each store's current inventory.
42     cout << "\nThe current inventory of each store:\n";
43     cout << "Store 1: " << store1 << endl;
44     cout << "Store 2: " << store2 << endl;
45     cout << "Store 3: " << store3 << endl;
46     return 0;
47 }
```

Program Output with Example Input Shown in Bold

```
One week ago, 3 new widget stores opened
at the same time with the same beginning
inventory. What was the beginning inventory? 100 Enter
```



```

How many widgets has store 1 sold? 25 
How many widgets has store 2 sold? 15 
How many widgets has store 3 sold? 45 

The current inventory of each store:
Store 1: 75
Store 2: 85
Store 3: 55

```

Table 3-10 shows other examples of such statements and their assignment statement equivalencies.

Table 3-10 Example Usage of the Combined Assignment Operators

Example Usage	Equivalent to
<code>x += b + 5;</code>	<code>x = x + (b + 5);</code>
<code>y -= a * 2;</code>	<code>y = y - (a * 2);</code>
<code>z *= 10 - c;</code>	<code>z = z * (10 - c);</code>
<code>a /= b + c;</code>	<code>a = a / (b + c);</code>
<code>c %= d - 3;</code>	<code>c = c % (d - 3);</code>

3.7 Formatting Output

المفهوم: يوفر كائن `cout` طرقاً لتنسيق البيانات أثناء عرضها.

هذا يؤثر على طريقة ظهور البيانات على الشاشة. يمكن طباعة أو عرض نفس البيانات بعدة طرق مختلفة. على سبيل المثال، جميع الأرقام التالية لها نفس القيمة، على الرغم من أنها تبدو مختلفة:

```

720
720.0
720.000000000
7.2e+2
+720.0

```

طريقة طباعة القيمة تُسمى **التنسيق**. كائن `cout` لديه طريقة قياسية لتنسيق المتغيرات من كل نوع بيانات. ومع ذلك، في بعض الأحيان تحتاج إلى تحكم أكبر في طريقة عرض البيانات. فكر في البرنامج 3-12، على سبيل المثال، الذي يعرض ثلاثة صفوف من الأرقام مع وجود مسافة بين كل منها.

Program 3-12

```

1 // This program displays three rows of numbers.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int num1 = 2897, num2 = 5,    num3 = 837,
8         num4 = 34,    num5 = 7,    num6 = 1623,
9         num7 = 390,    num8 = 3456, num9 = 12;

```



```

10
11 // Display the first row of numbers
12 cout << num1 << " " << num2 << " " << num3 << endl;
13
14 // Display the second row of numbers
15 cout << num4 << " " << num5 << " " << num6 << endl;
16
17 // Display the third row of numbers
18 cout << num7 << " " << num8 << " " << num9 << endl;
19 return 0;
20 }

```

Program Output

```

2897 5 837
34 7 1623
390 3456 12

```

للأسف، الأرقام لا تصطف في أعمدة. هذا لأن بعض الأرقام، مثل 5 و 7، تحتل موقعًا واحدًا على الشاشة، بينما تحتل أرقام أخرى موقعين أو ثلاثة. يستخدم `cout` فقط عدد المساحات اللازمة لطباعة كل رقم. لحل هذه المشكلة، يوفر `cout` طريقة لتحديد الحد الأدنى لعدد المساحات المستخدمة لكل رقم. يمكن استخدام أداة تنسيق الدفق `setw` لإنشاء حقول طباعة بعرض محدد. إليك مثال على كيفية استخدامها:

```
value = 23;
```

```
cout << setw(5) << value;
```

الرقم الموجود داخل الأقواس بعد كلمة `setw` يحدد عرض الحقل للقيمة التي تليها مباشرة. عرض الحقل هو الحد الأدنى لعدد المواقع أو المساحات على الشاشة لطباعة القيمة. في المثال أعلاه، سيتم عرض الرقم 23 في حقل مكون من 5 مساحات. وبما أن الرقم 23 يحتل موقعين فقط على الشاشة، سيتم طباعة 3 مساحات فارغة قبله. لتوضيح كيفية عمل ذلك، انظر إلى العبارات التالية:

```
value = 23;
```

```
cout << "(" << setw(5) << value << ")";
```

سيؤدي ذلك إلى الإخراج التالي:

```
( 23)
```

لاحظ أن الرقم يحتل الموضعين الأخيرين في الحقل. وبما أن الرقم لم يستخدم الحقل بالكامل، قام `cout` بملء المواضع الإضافية بمساحات فارغة. وبما أن الرقم يظهر على الجانب الأيمن من الحقل مع وجود مساحات فارغة "padding" أمامه، يُقال إنه محاذاة لليمين (*right-justified*). يوضح البرنامج 3-13 كيف يمكن طباعة الأرقام في البرنامج 3-12 في أعمدة تصطف بشكل مثالي باستخدام `setw`.

Program 3-13

```
1 // This program displays three rows of numbers.
2 #include <iostream>
3 #include <iomanip> // Required for setw
4 using namespace std;
5
6 int main()
7 {
8     int num1 = 2897, num2 = 5,    num3 = 837,
9         num4 = 34,    num5 = 7,    num6 = 1623,
10        num7 = 390,    num8 = 3456, num9 = 12;
11
12    // Display the first row of numbers
13    cout << setw(6) << num1 << setw(6)
14         << num2 << setw(6) << num3 << endl;
15
16    // Display the second row of numbers
17    cout << setw(6) << num4 << setw(6)
18         << num5 << setw(6) << num6 << endl;
19
20    // Display the third row of numbers
21    cout << setw(6) << num7 << setw(6)
22         << num8 << setw(6) << num9 << endl;
23    return 0;
24 }
```

Program Output

2897	5	837
34	7	1623
390	3456	12



NOTE: A new header file, `<iomanip>`, is included in Program 3-13. It must be used in any program that uses `setw`.

قد تتساءل عما سيحدث إذا كان الرقم كبيرًا جدًا بحيث لا يتناسب مع الحقل المحدد، كما في العبارة التالية:

```
value = 18397;
```

```
cout << setw(2) << value;
```


في مثل هذه الحالات، سيقوم cout بطباعة الرقم بالكامل. تحدد setw فقط الحد الأدنى لعدد المواضع في حقل الطباعة. أي رقم أكبر من هذا الحد الأدنى سيتسبب في تجاوز cout لقيمة setw. يمكنك تحديد عرض الحقل لأي نوع من البيانات. يوضح البرنامج 3-14 استخدام setw مع عدد صحيح، وعدد عشري، وكائن سلسلة نصية (string).

Program 3-14

```
1 // This program demonstrates the setw manipulator being
2 // used with values of various data types.
3 #include <iostream>
4 #include <iomanip>
5 #include <string>
6 using namespace std;
7
8 int main()
9 {
10     int intValue = 3928;
11     double doubleValue = 91.5;
12     string stringValue = "John J. Smith";
13
14     cout << "(" << setw(5) << intValue << ")" << endl;
15     cout << "(" << setw(8) << doubleValue << ")" << endl;
16     cout << "(" << setw(16) << stringValue << ")" << endl;
17     return 0;
18 }
```

Program Output

```
( 3928)
(    91.5)
(    John J. Smith)
```

يمكن استخدام البرنامج 3-14 لتوضيح النقاط التالية:

- عرض الحقل لعدد عشري يتضمن موقعاً للنقطة العشرية.
- عرض الحقل لكائن سلسلة نصية (string) يتضمن جميع الأحرف في السلسلة، بما في ذلك المسافات.
- القيم المطبوعة في الحقل تكون محاذاة لليمين بشكل افتراضي. هذا يعني أنها تتماشى مع الجانب الأيمن من حقل الطباعة، ويتم إدخال أي مسافات فارغة مطلوبة للتعبئة أمام القيمة.

• The setprecision Manipulator

يمكن تقريب القيم العشرية إلى عدد معين من الأرقام المعنوية، أو الدقة، وهو العدد الإجمالي للأرقام التي تظهر قبل وبعد النقطة العشرية. يمكنك التحكم في عدد الأرقام المعنوية التي يتم عرض القيم العشرية بها باستخدام أداة التنسيق setprecision. يوضح البرنامج 3-15 نتائج عملية قسمة معروضة بعدد مختلف من الأرقام المعنوية.

Program 3-15

```
1 // This program demonstrates how setprecision rounds a
2 // floating-point value.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main()
8 {
9     double quotient, number1 = 132.364, number2 = 26.91;
10
11     quotient = number1 / number2;
12     cout << quotient << endl;
13     cout << setprecision(5) << quotient << endl;
14     cout << setprecision(4) << quotient << endl;
15     cout << setprecision(3) << quotient << endl;
16     cout << setprecision(2) << quotient << endl;
17     cout << setprecision(1) << quotient << endl;
18     return 0;
19 }
```

Program Output

4.91877
4.9188
4.919
4.92
4.9
5

Table 3-11 The setprecision Manipulator

Number	Manipulator	Value Displayed
28.92786	setprecision(3)	28.9
21	setprecision(5)	21
109.5	setprecision(4)	109.5
34.28596	setprecision(2)	34

يوضح البرنامج 3-16 كيف يمكن الجمع بين أدوات التنسيق setw و setprecision للتحكم الكامل في طريقة عرض الأعداد العشرية.

Program 3-16

```
1 // This program asks for sales amounts for 3 days. The total
2 // sales are calculated and displayed in a table.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main()
8 {
9     double day1, day2, day3, total;
10
11     // Get the sales for each day.
12     cout << "Enter the sales for day 1: ";
13     cin >> day1;
14     cout << "Enter the sales for day 2: ";
15     cin >> day2;
16     cout << "Enter the sales for day 3: ";
17     cin >> day3;
18
19     // Calculate the total sales.
20     total = day1 + day2 + day3;
21
22     // Display the sales amounts.
23     cout << "\nSales Amounts\n";
24     cout << "-----\n";
25     cout << setprecision(5);
26     cout << "Day 1: " << setw(8) << day1 << endl;
27     cout << "Day 2: " << setw(8) << day2 << endl;
28     cout << "Day 3: " << setw(8) << day3 << endl;
29     cout << "Total: " << setw(8) << total << endl;
30     return 0;
31 }
```

Program Output with Example Input Shown in Bold

Enter the sales for day 1: **321.57**
Enter the sales for day 2: **269.62**
Enter the sales for day 3: **307.77**

Sales Amounts

Day 1: 321.57
Day 2: 269.62
Day 3: 307.77
Total: 898.96

- **The fixed Manipulator**

قد تفاجئك أداة التنسيق `setprecision` أحياناً بطريقة غير مرغوب فيها. عندما يتم ضبط دقة رقم على قيمة أقل، تميل الأرقام إلى الظهور بالتدوين العلمي. على سبيل المثال، إليك ناتج البرنامج 3-16 عند إدخال أرقام أكبر:

Program 3-16

Program Output with Example Input Shown in Bold

```
Enter the sales for day 1: 145678.99 
Enter the sales for day 2: 205614.85 
Enter the sales for day 3: 198645.22 

Sales Amounts
-----
Day 1: 1.4568e+005
Day 2: 2.0561e+005
Day 3: 1.9865e+005
Total: 5.4994e+005
```

أداة تنسيق أخرى، `fixed`، تجبر `cout` على طباعة الأرقام بتدوين النقطة الثابتة، أو العشري. يوضح البرنامج 3-17 كيفية استخدام أداة التنسيق `fixed`.

Program 3-17

```
1 // This program asks for sales amounts for 3 days. The total
2 // sales are calculated and displayed in a table.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 int main()
8 {
9     double day1, day2, day3, total;
10
11     // Get the sales for each day.
12     cout << "Enter the sales for day 1: ";
13     cin >> day1;
14     cout << "Enter the sales for day 2: ";
15     cin >> day2;
16     cout << "Enter the sales for day 3: ";
17     cin >> day3;
18
19     // Calculate the total sales.
20     total = day1 + day2 + day3;
21
```



```

22 // Display the sales amounts.
23 cout << "\nSales Amounts\n";
24 cout << "-----\n";
25 cout << setprecision(2) << fixed;
26 cout << "Day 1: " << setw(8) << day1 << endl;
27 cout << "Day 2: " << setw(8) << day2 << endl;
28 cout << "Day 3: " << setw(8) << day3 << endl;
29 cout << "Total: " << setw(8) << total << endl;
30 return 0;
31 }

```

Program Output with Example Input Shown in Bold

Enter the sales for day 1: **1321.87**

Enter the sales for day 2: **1869.26**

Enter the sales for day 3: **1403.77**

Sales Amounts

Day 1: 1321.87
Day 2: 1869.26
Day 3: 1403.77
Total: 4594.90

العبارة في السطر 25 تستخدم أداة التنسيق fixed:

cout << setprecision(2) << fixed;

عند استخدام أداة التنسيق 'fixed'، سيتم عرض جميع الأرقام العشرية التي يتم طباعتها لاحقاً بتدوين النقطة الثابتة، مع تحديد عدد الأرقام على يمين النقطة العشرية بواسطة أداة التنسيق 'setprecision'.

عند استخدام أداتي التنسيق 'fixed' و 'setprecision' معاً، ستكون القيمة المحددة بواسطة 'setprecision' هي عدد الأرقام التي تظهر بعد النقطة العشرية، وليس عدد الأرقام المعنوية. على سبيل المثال، انظر إلى الكود التالي:

double x = 123.4567;

cout << setprecision(2) << fixed << x << endl;

بسبب استخدام أداة التنسيق 'fixed'، ستجعل أداة التنسيق 'setprecision' الرقم يُعرض مع رقمين بعد النقطة العشرية. سيتم عرض القيمة كـ '123.46'.

• The showpoint Manipulator

بشكل افتراضي، لا يتم عرض الأرقام العشرية بالأصفار الزائدة، ولا يتم عرض الأرقام العشرية التي لا تحتوي على جزء كسري مع نقطة عشرية. على سبيل المثال، انظر إلى الكود التالي:

double x = 123.4, y = 456.0;

cout << setprecision(6) << x << endl;

cout << y << endl;

ستنتج عبارات 'cout' المخرجات التالية:

123.4

456

على الرغم من تحديد ستة أرقام معنوية لكلا الرقمين، لا يتم عرض أي منهما بأصفار زائدة. إذا كنا نريد أن يتم تعبئة الأرقام بأصفار زائدة، يجب علينا استخدام أداة التنسيق 'showpoint' كما هو موضح في الكود التالي:

```
double x = 123.4, y = 456.0;
cout << setprecision(6) << showpoint << x << endl;
cout << y << endl;
```

These **cout** statements will produce the following output:

```
123.400
456.000
```



NOTE: With most compilers, trailing zeros are displayed when the `setprecision` and `fixed` manipulators are used together.

• The left and right Manipulators

عادةً ما يكون الإخراج محاذاة لليمين. على سبيل المثال، انظر إلى الكود التالي:

```
double x = 146.789, y = 24.2, z = 1.783;
cout << setw(10) << x << endl;
cout << setw(10) << y << endl;
cout << setw(10) << z << endl;
```

يتم عرض كل من المتغيرات 'x' و 'y' و 'z' في حقل طباعة مكون من 10 مسافات. ناتج عبارات 'cout' هو:

```
146.789
 24.2
 1.783
```

لاحظ أن كل قيمة محاذاة لليمين، أو تتماشى مع الجانب الأيمن من حقل الطباعة الخاص بها. يمكنك جعل القيم محاذاة لليسر باستخدام أداة التنسيق 'left'، كما هو موضح في الكود التالي.

```
double x = 146.789, y = 24.2, z = 1.783;
cout << left << setw(10) << x << endl;
cout << setw(10) << y << endl;
cout << setw(10) << z << endl;
```

ناتج هذه العبارات 'cout' هو:

```
146.789
24.2
1.783
```

في هذه الحالة، يتم محاذاة الأرقام إلى الجانب الأيسر من حقول الطباعة الخاصة بها. تظل أداة التنسيق 'left' سارية المفعول حتى تستخدم أداة التنسيق 'right'، والتي تجعل كل الإخراج اللاحق محاذاة لليمين.

Table 3-12 Stream Manipulators

Stream Manipulator	Description
<code>setw(n)</code>	Establishes a print field of <i>n</i> spaces.
<code>fixed</code>	Displays floating-point numbers in fixed-point notation.
<code>showpoint</code>	Causes a decimal point and trailing zeros to be displayed, even if there is no fractional part.
<code>setprecision(n)</code>	Sets the precision of floating-point numbers.
<code>left</code>	Causes subsequent output to be left-justified.
<code>right</code>	Causes subsequent output to be right-justified.

3.8 Working with Characters and string Objects

المفهوم: توجد وظائف خاصة للتعامل مع الأحرف وكائنات السلاسل النصية.

على الرغم من أنه يمكن استخدام `cin` مع العامل `>>` لإدخال السلاسل النصية، إلا أن ذلك قد يسبب مشاكل يجب أن تكون على علم بها. عندما يقرأ `cin` المدخلات، يتخطى ويتجاهل أي مسافات بيضاء في البداية (مسافات، علامات تبويب، أو فواصل أسطر). بمجرد أن يصل إلى الحرف الأول غير الفارغ ويبدأ القراءة، يتوقف عن القراءة عندما يصل إلى المسافة البيضاء التالية. يوضح البرنامج 3-18 هذه المشكلة.

Program 3-18

```
1 // This program illustrates a problem that can occur if
2 // cin is used to read character data into a string object.
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 int main()
8 {
9     string name;
10    string city;
11
12    cout << "Please enter your name: ";
13    cin >> name;
14    cout << "Enter the city you live in: ";
15    cin >> city;
16
17    cout << "Hello, " << name << endl;
18    cout << "You live in " << city << endl;
19    return 0;
20 }
```

Program Output with Example Input Shown in Bold

```
Please enter your name: Kate Smith 
Enter the city you live in: Hello, Kate
You live in Smith
```

لاحظ أن المستخدم لم يُمنح الفرصة لإدخال المدينة. في عبارة الإدخال الأولى، عندما وصل `cin` إلى المسافة بين `Kate` و `Smith`، توقف عن القراءة، وقام بتخزين `Kate` فقط كقيمة للمتغير `name`. في عبارة الإدخال الثانية، استخدم `cin` الأحرف المتبقية الموجودة في مخزن لوحة المفاتيح وقام بتخزين `Smith` كقيمة للمتغير `city`.

لحل هذه المشكلة، يمكنك استخدام دالة في C++ تُسمى `getline`. تقوم دالة `getline` بقراءة سطر كامل، بما في ذلك المسافات في البداية والمدمجة، وتخزينه في كائن سلسلة نصية (string). تبدو دالة `getline` كما يلي، حيث `cin` هو كائن الإدخال الذي نقرأ منه، و `inputLine` هو اسم كائن السلسلة النصية الذي يستقبل المدخلات:

```
getline(cin, inputLine);
```


Program 3-19

```
1 // This program demonstrates using the getline function
2 // to read character data into a string object.
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 int main()
8 {
9     string name;
10    string city;
11
12    cout << "Please enter your name: ";
13    getline(cin, name);
14    cout << "Enter the city you live in: ";
15    getline(cin, city);
16
17    cout << "Hello, " << name << endl;
18    cout << "You live in " << city << endl;
19    return 0;
20 }
```

Program Output with Example Input Shown in Bold

```
Please enter your name: Kate Smith 
Enter the city you live in: Raleigh 
Hello, Kate Smith
You live in Raleigh
```

- **Inputting a Character**

في بعض الأحيان، قد ترغب في قراءة حرف واحد فقط من المدخلات. على سبيل المثال، تعرض بعض البرامج قائمة عناصر ليختار المستخدم منها. غالبًا ما يُشار إلى الخيارات بالأحرف A، B، C، وهكذا. يقوم المستخدم باختيار عنصر من القائمة عن طريق كتابة حرف.

أبسط طريقة لقراءة حرف واحد هي باستخدام 'cin' والعامل '>>'، كما هو موضح في البرنامج 3-20.

Program 3-20

```
1 // This program reads a single character into a char variable.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     char ch;
8
9     cout << "Type a character and press Enter: ";
10    cin >> ch;
11    cout << "You entered " << ch << endl;
12    return 0;
13 }
```

Program Output with Example Input Shown in Bold

```
Type a character and press Enter: A 
You entered A
```


• Using cin.get

كما هو الحال مع إدخال السلاسل النصية، هناك أوقات لا يقوم فيها استخدام `cin>>` لقراءة حرف بما تريده. على سبيل المثال، لأنه يتخطى جميع المسافات البيضاء في البداية، يكون من المستحيل إدخال مسافة بسيطة أو الضغط على Enter باستخدام `cin>>`. لن يستمر البرنامج بعد عبارة `cin` حتى يتم الضغط على حرف غير مفتاح المسافة، مفتاح Tab، أو مفتاح Enter. (بمجرد إدخال مثل هذا الحرف، يجب الضغط على مفتاح Enter قبل أن يتمكن البرنامج من الانتقال إلى العبارة التالية.) وبالتالي، البرامج التي تطلب من المستخدم "اضغط على مفتاح Enter للمتابعة" لا يمكنها استخدام العامل `>>` لقراءة الضغط على مفتاح Enter فقط.

في مثل هذه الحالات، يحتوي كائن `cin` على دالة مدمجة تسمى `get` تكون مفيدة. لأن دالة `get` مدمجة في كائن `cin`، نقول إنها دالة عضو في `cin`. تقوم دالة العضو `get` بقراءة حرف واحد، بما في ذلك أي حرف مسافة بيضاء. إذا كان البرنامج بحاجة إلى تخزين الحرف الذي يتم قراءته، يمكن استدعاء دالة العضو `get` بإحدى الطريقتين التاليتين. في كلا المثالين، افترض أن `ch` هو اسم متغير من نوع `char` يتم فيه تخزين الحرف المقروء.

```
cin.get(ch);  
ch = cin.get();
```

إذا كان البرنامج يستخدم دالة `cin.get` فقط لإيقاف الشاشة مؤقتًا حتى يتم الضغط على مفتاح Enter ولا يحتاج إلى تخزين الحرف، يمكن أيضًا استدعاء الدالة بهذه الطريقة:

```
cin.get();
```

يوضح البرنامج 3-21 جميع الطرق الثلاث لاستخدام دالة `cin.get`.

Program 3-21

```
1 // This program demonstrates three ways  
2 // to use cin.get() to pause a program.  
3 #include <iostream>  
4 using namespace std;  
5  
6 int main()  
7 {  
8     char ch;  
9  
10    cout << "This program has paused. Press Enter to continue."  
11    cin.get(ch);  
12    cout << "It has paused a second time. Please press Enter again."  
13    ch = cin.get();  
14    cout << "It has paused a third time. Please press Enter again."  
15    cin.get();  
16    cout << "Thank you!";  
17    return 0;  
18 }
```

Program Output with Example Input Shown in Bold

```
This program has paused. Press Enter to continue. Enter  
It has paused a second time. Please press Enter again. Enter  
It has paused a third time. Please press Enter again. Enter  
Thank you!
```


- **Mixing cin >> and cin.get**

يمكن أن يؤدي الجمع بين `cin>>` و `cin.get` إلى حدوث مشكلة مزعجة ويصعب اكتشافها. على سبيل المثال، انظر إلى البرنامج 3-22.

Program 3-22

```
1 // This program demonstrates a problem that occurs
2 // when you mix cin >> with cin.get().
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     char ch;                // Define a character variable
9     int number;             // Define an integer variable
10
```

Program 3-22 (continued)

```
11     cout << "Enter a number: ";
12     cin >> number;          // Read an integer
13     cout << "Enter a character: ";
14     ch = cin.get();          // Read a character
15     cout << "Thank You!\n";
16     return 0;
17 }
```

Program Output with Example Input Shown in Bold

```
Enter a number: 100 Enter
Enter a character: Thank You!
```

عند تشغيل هذا البرنامج، يسمح السطر 12 للمستخدم بإدخال رقم، ولكن يبدو وكأن العبارة في السطر 14 تم تخطيها. يحدث هذا لأن `cin>>` و `cin.get` يستخدمان تقنيات مختلفة قليلاً لقراءة البيانات.

في المثال التشغيلي للبرنامج، عند تنفيذ السطر 12، أدخل المستخدم الرقم 100 واضغط على مفتاح Enter. يؤدي الضغط على مفتاح Enter إلى تخزين حرف سطر جديد (`\n`) في مخزن لوحة المفاتيح (keyboard buffer)، كما هو موضح في الشكل 3-5. تبدأ عبارة `cin>>` في السطر 12 بقراءة البيانات التي أدخلها المستخدم، وتتوقف عن القراءة عندما تصل إلى حرف السطر الجديد. هذا موضح في الشكل 3-6. لا يتم قراءة حرف السطر الجديد، ولكنه يبقى في مخزن لوحة المفاتيح.

Figure 3-5 Contents of the keyboard buffer

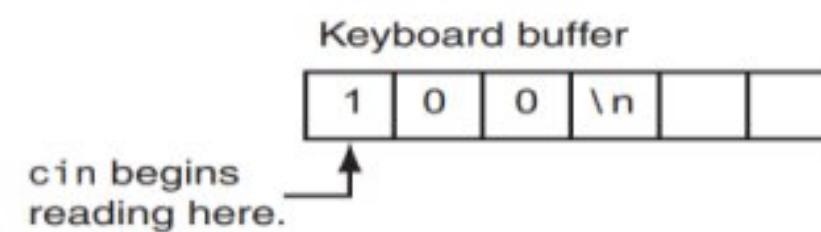
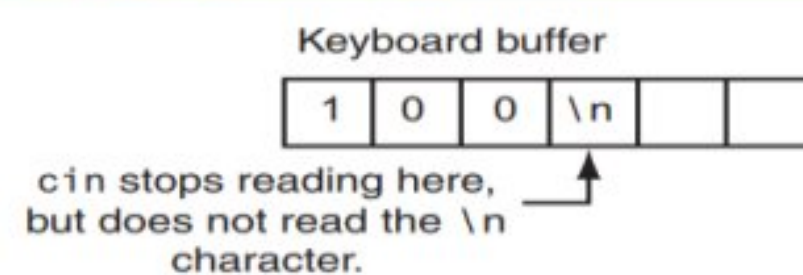


Figure 3-6 cin stops reading at the newline character



عند تنفيذ دالة `cin.get` في السطر 14، تبدأ بقراءة مخزن لوحة المفاتيح (keyboard buffer) حيث توقفت عملية الإدخال السابقة. هذا يعني أن `cin.get` تقرأ حرف السطر الجديد، دون منح المستخدم فرصة لإدخال أي بيانات إضافية. يمكنك معالجة هذه الحالة باستخدام دالة `cin.ignore`، والتي سيتم شرحها في القسم التالي.

- **Using cin.ignore**

لحل المشكلة الموصوفة سابقاً، يمكنك استخدام دالة أخرى من دوال العضو في كائن `cin` تسمى `ignore`. تخبر دالة `cin.ignore` كائن `cin` بتخطي حرف واحد أو أكثر في مخزن لوحة المفاتيح. إليك شكلها العام:

cin.ignore(n, c);

الوسائط الموضحة بين الأقواس اختيارية. إذا تم استخدامها، فإن `n` هو عدد صحيح و `c` هو حرف. تخبر هذه الوسائط `cin` بتخطي عدد `n` من الأحرف، أو حتى يتم الوصول إلى الحرف `c`. على سبيل المثال، العبارة التالية تجعل `cin` تتخطى الـ 20 حرفاً التالية أو حتى يتم الوصول إلى حرف سطر جديد، أيهما يأتي أولاً:

cin.ignore(20, '\n');

إذا لم يتم استخدام أي وسائط، فإن `cin` ستتخطى الحرف التالي فقط. إليك مثال:

cin.ignore();

يوضح البرنامج 3-23، وهو نسخة معدلة من البرنامج 3-22، كيفية استخدام الدالة. لاحظ أنه تم إدراج استدعاء لـ `cin.ignore` في السطر 13، مباشرة بعد عبارة `cin>>`.

Program 3-23

```
1 // This program successfully uses both
2 // cin >> and cin.get() for keyboard input.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     char ch;
9     int number;
10
11     cout << "Enter a number: ";
12     cin >> number;
13     cin.ignore(); // Skip the newline character
14     cout << "Enter a character: ";
15     ch = cin.get();
16     cout << "Thank You!\n";
17     return 0;
18 }
```

Program Output with Example Input Shown in Bold

Enter a number: **100**
Enter a character: **Z**
Thank You!

- **string Member Functions and Operators**

تحتوي كائنات السلسلة النصية (string) في C++ أيضاً على عدد من دوال العضو. على سبيل المثال، إذا كنت تريد معرفة طول السلسلة النصية المخزنة في كائن 'string'، يمكنك استدعاء دالة العضو 'length'. إليك مثال على كيفية استخدامها:

```
string state = "Texas";
```

```
int size = state.length();
```

العبارة الأولى تنشئ كائن 'string' باسم 'state' وتقوم بتهيئته بالسلسلة النصية "Texas". العبارة الثانية تُعرّف متغيراً من نوع 'int' باسم 'size' وتقوم بتهيئته بطول السلسلة النصية في كائن 'state'. بعد تنفيذ هذا الكود، سيحتوي المتغير 'size' على القيمة 5.

بعض العوامل تعمل أيضاً مع كائنات السلسلة النصية. أحدها هو العامل '+'، لقد صادفت بالفعل العامل '+' لإضافة كميتين رقميتين. نظراً لأنه لا يمكن إضافة السلاسل النصية، عندما يتم استخدام هذا العامل مع معاملات من نوع 'string'، فإنه يقوم بدمجها أو ربطها معاً. افترض أن لدينا التعريفات والتهيئات التالية في برنامج:

```
string greeting1 = "Hello ";
```

```
string greeting2;
```

```
string name1 = "World";
```

```
string name2 = "People";
```

توضح العبارات التالية كيفية عمل دمج السلاسل النصية:

```
greeting2 = greeting1 + name1; // greeting2 now holds "Hello World"
```

```
greeting1 = greeting1 + name2; // greeting1 now holds "Hello People"
```

لاحظ أن السلسلة النصية المخزنة في 'greeting1' تحتوي على مسافة كحرفها الأخير. إذا لم تكن المسافة موجودة، لكانت 'greeting2' قد أعطيت السلسلة النصية "HelloWorld".

العبارة الأخيرة في المثال السابق يمكن أيضاً كتابتها باستخدام عامل التعيين المركب '+='. لتحقيق نفس النتيجة:

```
greeting1 += name2; // greeting1 now holds "Hello People"
```

3.9 More Mathematical Library Functions

المفهوم: توفر مكتبة وقت تشغيل C++ العديد من الدوال لإجراء العمليات الرياضية المعقدة.

في وقت سابق من هذه المحاضرة، تعلمت استخدام دالة 'pow' لرفع رقم إلى قوة معينة. تحتوي مكتبة C++ على العديد من الدوال الأخرى التي تقوم بعمليات رياضية متخصصة. هذه الدوال مفيدة في البرامج العلمية والبرامج ذات الأغراض الخاصة. يوضح الجدول 3-13 بعضاً من هذه الدوال، كل منها يتطلب تضمين ملف الرأس '<cmath>'.

Table 3-13 <cmath> Library Functions

Function	Example	Description
abs	<code>y = abs(x);</code>	Returns the absolute value of the argument. The argument and the return value are integers.
cos	<code>y = cos(x);</code>	Returns the cosine of the argument. The argument should be an angle expressed in radians. The return type and the argument are doubles.
exp	<code>y = exp(x);</code>	Computes the exponential function of the argument, which is x. The return type and the argument are doubles.
fmod	<code>y = fmod(x, z);</code>	Returns, as a double, the remainder of the first argument divided by the second argument. Works like the modulus operator, but the arguments are doubles. (The modulus operator only works with integers.) Take care not to pass zero as the second argument. Doing so would cause division by zero.
log	<code>y = log(x);</code>	Returns the natural logarithm of the argument. The return type and the argument are doubles.
log10	<code>y = log10(x);</code>	Returns the base-10 logarithm of the argument. The return type and the argument are doubles.
round	<code>y = round(x)</code>	The argument, x, can be a double, a float, or a long double. Returns the value of x rounded to the nearest whole number. For example, if x is 2.8, the function returns 3.0, or if x is 2.1, the function returns 2.0. The return type is the same as the type of the argument.
sin	<code>y = sin(x);</code>	Returns the sine of the argument. The argument should be an angle expressed in radians. The return type and the argument are doubles.
sqrt	<code>y = sqrt(x);</code>	Returns the square root of the argument. The return type and argument are doubles.
tan	<code>y = tan(x);</code>	Returns the tangent of the argument. The argument should be an angle expressed in radians. The return type and the argument are doubles.

كل من هذه الدوال بسيط الاستخدام مثل دالة `pow`. يوضح مقطع البرنامج التالي دالة `sqrt`، التي تُرجع الجذر التربيعي لرقم معين:

```
cout << "Enter a number: ";
cin >> num;
s = sqrt(num);
cout << "The square root of " << num << " is " << s << endl;
```

إليك ناتج مقطع البرنامج، مع إدخال المستخدم للرقم 25:

```
Enter a number: 25
The square root of 25 is 5
```

يوضح البرنامج 3-24 استخدام دالة `sqrt` لإيجاد طول الوتر في مثلث قائم الزاوية. يستخدم البرنامج الصيغة التالية، المأخوذة من نظرية فيثاغورس:

Program 3-24

```
1 // This program asks for the lengths of the two sides of a
2 // right triangle. The length of the hypotenuse is then
3 // calculated and displayed.
4 #include <iostream>
5 #include <iomanip> // For setprecision
6 #include <cmath> // For the sqrt and pow functions
7 using namespace std;
8
9 int main()
10 {
11     double a, b, c;
12
13     cout << "Enter the length of side a: ";
14     cin >> a;
15     cout << "Enter the length of side b: ";
16     cin >> b;
17     c = sqrt(pow(a, 2.0) + pow(b, 2.0));
18     cout << "The length of the hypotenuse is ";
19     cout << setprecision(2) << c << endl;
20     return 0;
21 }
```

Program Output with Example Input Shown in Bold

Enter the length of side a: **5.0**
Enter the length of side b: **12.0**
The length of the hypotenuse is 13

• Random Numbers

- الأرقام العشوائية مفيدة للعديد من المهام البرمجية المختلفة. فيما يلي بعض الأمثلة فقط:
- تُستخدم الأرقام العشوائية بشكل شائع في الألعاب. على سبيل المثال، الألعاب الإلكترونية التي تتيح للاعب رمي النرد تستخدم الأرقام العشوائية لتمثيل قيم النرد.
- البرامج التي تعرض بطاقات يتم سحبها من مجموعة أوراق مخلوطة تستخدم أرقامًا عشوائية لتمثيل القيم الظاهرة للبطاقات.
- الأرقام العشوائية مفيدة في برامج المحاكاة. في بعض المحاكاة، يجب على الكمبيوتر أن يقرر بشكل عشوائي كيفية تصرف شخص أو حيوان أو حشرة أو كائن حي آخر. يمكن إنشاء صيغ تُستخدم فيها الأرقام العشوائية لتحديد الإجراءات والأحداث المختلفة التي تحدث في البرنامج.
- الأرقام العشوائية مفيدة في البرامج الإحصائية التي يجب أن تختار البيانات بشكل عشوائي لتحليلها.
- تُستخدم الأرقام العشوائية بشكل شائع في أمن الكمبيوتر لتشفير البيانات الحساسة.

توفر مكتبة لغة C++ كل ما تحتاجه لتوليد أرقام عشوائية في برامجك. لاستخدام هذه الإمكانيات، ستحتاج إلى التوجيه التالي:

#include <random>

- توليد أرقام عشوائية، ستحتاج إلى تعريف كائن في برنامجك:
- محرك توليد الأرقام العشوائية (Random Number Engine)
- كائن التوزيع (Distribution Object)

يقوم محرك توليد الأرقام العشوائية بإنشاء سلسلة من البتات العشوائية. بينما يقوم كائن التوزيع بقراءة هذه البتات العشوائية التي تم إنشاؤها بواسطة المحرك وإنتاج أرقام عشوائية من نوع بيانات محدد، ضمن نطاق معين.

توفر مكتبة C++ عدة محركات لتوليد الأرقام العشوائية التي يمكنك استخدامها بناءً على احتياجات تطبيقك. سنستخدم بشكل أساسي محرك `random\_device`، والذي يعتبر مناسباً لمعظم التطبيقات. إليك مثالاً يوضح كيفية تعريف محرك `random\_device` باسم `myEngine`:

#include <random> // تتضمن مكتبة توليد الأرقام العشوائية

using namespace std;

int main()

{

random_device myEngine; // تعريف محرك توليد الأرقام العشوائية

return 0;

}

في هذا المثال، تم إنشاء كائن `myEngine` من نوع `random\_device`، والذي يمكن استخدامه لاحقاً لتوليد أرقام عشوائية.

بعد ذلك، يجب أن تحدد كائن توزيع. إذا كنت تريد أعداداً صحيحة عشوائية، يمكنك تعريف كائن `uniform\_int\_distribution`. إليك مثال على ذلك:

uniform_int_distribution<int> randomInt(0, 100);

هذا البيان يعرف كائن توزيع باسم `randomInt`. لاحظ كلمة `int` التي تظهر داخل الأقواس الزاوية (< >). هذا يحدد أن الكائن سيولد أرقاماً من نوع البيانات `int`. يمكنك تحديد أيًا من أنواع الأعداد الصحيحة في C++ (مثل `short`، `int`، `long`، وما إلى ذلك).

الأرقام `0` و `100` التي تظهر داخل الأقواس تحدد القيم الدنيا والقصوى للعدد العشوائي. في هذا المثال، سيقوم كائن التوزيع بتوليد أرقام في النطاق من `0` إلى `100`.

بافتراض أننا قمنا بتعريف محرك أرقام عشوائية باسم `myEngine` وكائن توزيع باسم `randomInt`، فإن البيان التالي سيقوم بتوليد عدد عشوائي وتعيينه للمتغير `number`:

int number = randomInt(myEngine);

البيان التالي سيقوم بتوليد عدد عشوائي وعرضه على الشاشة:

cout << randomInt(myEngine) << endl;

يمكنك أيضاً توليد أرقام عشوائية ذات فاصلة عشرية (نوع الأعداد الحقيقية) عن طريق تعريف كائن من النوع `uniform\_real\_distribution`. إليك مثال على ذلك:

uniform_real_distribution <double> randomReal(1.0 ,0.0);

في هذا المثال، سيتم توليد أرقام حقيقية عشوائية في النطاق بين `0.0` و `1.0`.

بافتراض أننا قمنا بتعريف محرك أرقام عشوائية باسم myEngine وكائن توزيع باسم randomReal، فإن البيان التالي سيقوم بتوليد عدد عشوائي ذو فاصلة عشرية) نوع floating-point وتعيينه (إسناده) للمتغير number:

double number = randomReal(myEngine);

إليك البرنامج 3-25 الذي يقوم بمحاكاة إلقاء مكعب النرد ذي الوجوه الستة:

Program 3-25

```
1 // This program simulates rolling dice.
2 #include <iostream>
3 #include <random>
4 using namespace std;
5
6 int main()
7 {
8     // Constants
9     const int MIN = 1;    // Minimum dice value
10    const int MAX = 6;    // Maximum dice value
11
12    // Random number engine
13    random_device engine;
14
15    // Distribution object
16    uniform_int_distribution<int> diceValue(MIN, MAX);
17
18    cout << "Rolling the dice...\n";
19    cout << diceValue(engine) << endl;
20    cout << diceValue(engine) << endl;
21
22    return 0;
23 }
```

Program Output

Rolling the dice...

5

2

Program Output

Rolling the dice...

4

6

Program Output

Rolling the dice...

3

1

3.10 Focus on Debugging: Hand Tracing a Program

تتبع البرنامج يدويًا هو عملية تصحيح (debugging) حيث تتظاهر بأنك الكمبيوتر الذي ينفذ البرنامج. تقوم بمراجعة كل تعليمة من تعليمات البرنامج واحدة تلو الأخرى. أثناء النظر إلى كل تعليمة، تقوم بتسجيل المحتويات التي ستكون عليها كل متغير بعد تنفيذ التعليمة. هذه العملية غالبًا ما تكون مفيدة في اكتشاف الأخطاء الرياضية وأخطاء المنطق الأخرى.

لنتتبع برنامج يدويًا، نقوم بإنشاء جدول يحتوي على عمود لكل متغير. الصفوف في الجدول تتوافق مع الأسطر في البرنامج. على سبيل المثال، البرنامج 3-26 موضح مع جدول تتبع يدوي. يستخدم البرنامج المتغيرات الأربعة التالية: `num1`، `num2`، `num3`، و `avg`. لاحظ أن جدول التتبع اليدوي يحتوي على عمود لكل متغير، وصف لكل سطر من الأكواد في الدالة الرئيسية (`main`).

Program 3-26

```
1 // This program asks for three numbers, then
2 // displays the average of the numbers.
3 #include <iostream>
4 using namespace std;

5 int main()
6 {
7     double num1, num2, num3, avg;
8     cout << "Enter the first number: ";
9     cin >> num1;
10    cout << "Enter the second number: ";
11    cin >> num2;
12    cout << "Enter the third number: ";
13    cin >> num3;
14    avg = num1 + num2 + num3 / 3;
15    cout << "The average is " << avg << endl;
16    return 0;
17 }
```

num1	num2	num3	avg

هذا البرنامج، الذي يطلب من المستخدم إدخال ثلاثة أرقام ثم يعرض متوسط هذه الأرقام، يحتوي على خطأ (bug). فهو لا يعرض المتوسط الصحيح. فيما يلي ناتج تنفيذ عينة للبرنامج:

Program Output with Example Input Shown in Bold

```
Enter the first number: 10 Enter
Enter the second number: 20 Enter
Enter the third number: 30 Enter
The average is 40
```

المتوسط الصحيح للأرقام 10 و 20 و 30 هو 20، وليس 40. للعثور على الخطأ، سنقوم بتتبع البرنامج يدويًا. لنتتبع هذا البرنامج يدويًا، نقوم بمراجعة كل تعليمة، ومراقبة العملية التي تتم، ثم تسجيل محتويات المتغيرات بعد تنفيذ التعليمة. بعد الانتهاء من التتبع اليدوي، سيظهر الجدول كما يلي. لقد وضعنا علامات استفهام (?) في الجدول حيث لا نعرف محتويات المتغير.

Program 3-26 (with hand trace chart filled)

```
1 // This program asks for three numbers, then
2 // displays the average of the numbers.
3 #include <iostream>
4 using namespace std;
5 int main()
6 {
7     double num1, num2, num3, avg;
8     cout << "Enter the first number: ";
9     cin >> num1;
10    cout << "Enter the second number: ";
11    cin >> num2;
12    cout << "Enter the third number: ";
13    cin >> num3;
14    avg = num1 + num2 + num3 / 3;
15    cout << "The average is " << avg << endl;
16    return 0;
17 }
```

num1	num2	num3	avg
?	?	?	?
?	?	?	?
10	?	?	?
10	?	?	?
10	20	?	?
10	20	?	?
10	20	30	?
10	20	30	40
10	20	30	40

هل ترى الخطأ؟ من خلال فحص التعليمة التي تقوم بالعملية الحسابية في السطر 14، نجد خطأً. عملية القسمة تتم قبل عمليات الجمع، لذا يجب علينا إعادة كتابة هذه التعليمة على النحو التالي:

avg = (num1 + num2 + num3) / 3;

تتبع البرنامج يدوياً هو عملية بسيطة تركز انتباهك على كل تعليمة في البرنامج. غالباً ما يساعدك هذا في اكتشاف الأخطاء التي ليست واضحة.

انتهت المحاضرة الثالثة